

College Social Network Answer

ROLE

Act as a senior QA automation engineer, SDET, test architect, requirements analyst, and debugging specialist working directly inside the current VS Code workspace.

You are completing an AI-assisted UI automation assessment for the **College Social Network** web application.

Your implementation technology is strictly:

- **Playwright**
- **TypeScript**
- Page Object Model
- Reusable fixtures/helpers/utilities
- Playwright assertions
- Integrated VS Code terminal for execution

Your objective is NOT merely to generate code.

Your objective is to:

inspect → analyze → document → derive requirements → identify validation rules → generate comprehensive test cases → implement automation → execute tests → investigate failures → fix automation issues → re-run → identify genuine application defects → document actual defects → perform final regression verification.

The final workspace must contain a complete, executable, maintainable solution satisfying every assessment requirement.

You are assisting me with the **College Social Network assignment**. Follow the assignment exactly and work with the **existing workspace**.

Analyse the existing project workspace first. Do NOT change, delete, rename, or restructure the provided workspace. Implement the solution on top of the existing structure only. Avoid redundant files/folders.

Use Playwright with TypeScript and Page Object Model. Use the integrated VS Code terminal for installation, execution, debugging, and validation.

The assessment evaluates both:

1. What was analysed/discovered and shared with AI: application analysis, login flow, element IDs, field types, acceptance criteria, and validation rules.
2. How AI prompts were structured: research, generation, code-generation, and refinement/follow-up prompts.

Perform the following steps in order.

APPLICATION UNDER TEST

System: College Social Network

URL: <http://webapps.tekstac.com:2121/>

Username: admin

Password: admin123

Navigation after login:

Student menu → Register

Student menu → Update Backlog

Faculty menu → Add Faculty

SCOPE

Module 1 — Student Registration: US-01 to US-08

Module 2 — Update Backlog: US-09 to US-11

Module 3 — Add Faculty: US-12 to US-16

Read the user stories/use cases already available in the workspace and treat them as the source of truth.

STEP 1 — LOGIN AND INSPECT ALL THREE MODULES

Login with admin/admin123 and inspect every form.

Identify and document:

- HTML element IDs
- Labels and field types
- Buttons/navigation
- Default values
- Success/error messages
- Form behaviour
- Mandatory fields
- Client/application validation
- Empty-form behaviour
- Invalid-value behaviour

Actually submit empty and invalid data to observe the application. Do not assume behaviour.

STEP 2 — DEFINE ACCEPTANCE CRITERIA

Create acceptance criteria for every user story US-01 through US-16 using the existing user stories plus actual application behaviour.

Create:

Application Analysis & Requirements.txt

It MUST contain:

- Application overview

- Login credentials/flow
- Navigation flow
- Module analysis
- US-01 through US-16
- Acceptance criteria
- Element IDs
- Field types
- Actual observed behaviour
- Validation rules
- Success/error messages
- Any discrepancies discovered

Use this file as context for subsequent AI test-case and code generation.

STEP 3 — RESEARCH VALIDATION RULES

Document all field-level validation rules for all three modules in Application Analysis & Requirements.txt.

Required rules:

Student Registration:

- Student Name: letters and spaces only
- Mobile Number: exactly 10 digits
- Email ID: valid email format
- CGPA: 0.0 to 10.0
- Department Name: non-empty text
- Backlog Count: non-negative integer
- All 6 fields mandatory

Update Backlog:

- Roll Number: valid roll number from a completed student registration
- Backlog Count: non-negative integer
- Both fields mandatory

Add Faculty:

- Faculty Name: letters and spaces only
- Email ID: valid email format
- Designation: non-empty text
- All 3 fields mandatory

Also document any additional rules discovered during actual inspection.

STEP 4 — GENERATE COMPREHENSIVE TEST CASES

Generate AT LEAST 60 detailed test cases covering ALL 16 user stories across all 3 modules.

Create a CSV containing exactly:

TC_ID, Module, User_Story_ID, Test_Scenario, Test_Type, Priority, Precondition, Test_Steps, Test_Data, Expected_Result, Actual_Result, Status

Requirements:

- Every US-01 through US-16 must be covered.
- Include positive, negative, validation, boundary, mandatory-field, invalid-format and functional scenarios.
- 60–70% MUST be negative/validation tests.
- Negative/validation tests should be Critical or High priority.
- Positive/success tests should be Low priority.
- Boundary tests should be Medium priority.
- Allowed priorities: Critical, High, Medium, Low.
- Use actual validation rules, element IDs and observed behaviour.
- Do not invent behaviour.
- Update Actual_Result and Status after execution where applicable.

STEP 5 — AUTOMATION IMPLEMENTATION

Create a complete Playwright TypeScript automation solution using Page Object Model.

Required:

- Browser setup/configuration
- Reusable login helper/page
- Student Registration page object
- Update Backlog page object
- Add Faculty page object
- Test methods
- Reusable utilities where needed
- Meaningful assertions
- Reliable Playwright locators
- Proper waits/synchronization

Implement AT LEAST 20 automated test methods covering all 3 modules.

Prioritize:

1. Critical validations
2. High-risk negative tests
3. Boundary tests
4. Positive scenarios

Every module must login using:

admin / admin123

Use the actual element IDs and validation rules discovered during inspection. Do not guess selectors.

STEP 6 — EXECUTE AND REFINE

Run the Playwright tests using the integrated VS Code terminal.

Do not stop at the first execution.

For every failure:

1. Inspect the actual error.
2. Determine whether it is a locator, navigation, timing, test-data, assertion, application-behaviour, or genuine application-defect issue.
3. Fix automation issues.
4. Correct assertions only when actual application behaviour proves the original assertion was wrong.
5. Re-run failed tests.
6. Re-run the complete suite where practical.
7. Improve stability and coverage.

Do NOT remove assertions or weaken valid validations just to make tests pass.

Provide execution evidence through Playwright reports/artifacts where supported.

STEP 7 — DOCUMENT DEFECTS

Execute the validation tests across ALL THREE modules and document defects based ONLY on actual observed execution.

Create exactly ONE consolidated file:

defect_report.json

It must contain AT LEAST 8 documented validation defects.

Each defect must include at minimum:

- Defect ID
- Module
- User Story ID
- Test Case ID
- Test Data / What was entered
- Steps/scenario
- Expected Result
- Actual Result
- Severity
- Priority
- Status

Defects MUST come from actual test execution, not AI prediction or assumptions.

Attempt to identify defects across all three modules. Execute enough negative/boundary/validation tests to obtain genuine observations. Do not fabricate defects merely to reach eight.

FINAL DELIVERABLES

The workspace must contain:

1. Application Analysis & Requirements.txt
 - Analysis of all 3 modules
 - Login flow
 - Element IDs
 - Field types
 - US-01 to US-16
 - Acceptance criteria
 - Validation rules
 - Actual observed behaviour
2. Test case CSV
 - 60+ detailed cases
 - All 16 user stories
 - Required columns
 - 60–70% negative/validation coverage
 - Correct priorities
3. Playwright TypeScript automation codebase
 - POM
 - Login helper
 - Student Registration page
 - Update Backlog page
 - Add Faculty page
 - At least 20 automated tests
 - Meaningful assertions
4. defect_report.json
 - One consolidated JSON file
 - At least 8 actual validation defects
 - Expected result
 - Actual result
 - Severity
 - Priority
 - Execution details

MINIMUM PASSING EXPECTATIONS

Before finishing, verify every item:

- Application Analysis & Requirements.txt exists with documented findings.
- 60+ test cases exist covering all 16 user stories.
- Test coverage is risk-based with 60–70% negative/validation scenarios.
- At least 20 automated tests exist with meaningful assertions.
- Login helper exists.
- Page objects exist for Student Registration, Update Backlog, and Add Faculty.
- All three modules are automated.
- Tests use admin/admin123 login.
- Tests have actually been executed.

- Failed tests have been investigated and refined.
- defect_report.json exists.
- defect_report.json contains at least 8 documented defects.
- Defects are based on actual test execution.
- Every defect includes expected result, actual result, severity, and priority.
- Existing workspace structure remains unchanged.
- No unnecessary/redundant files or folders are created.
- All files are saved and ready for evaluation.

IMPORTANT:

Do not merely explain what should be done. Perform the analysis, inspect the application, create the required files, generate the test cases, implement the Playwright TypeScript framework, execute the tests, debug/refine failures, document actual defects, and leave the completed solution in the workspace.